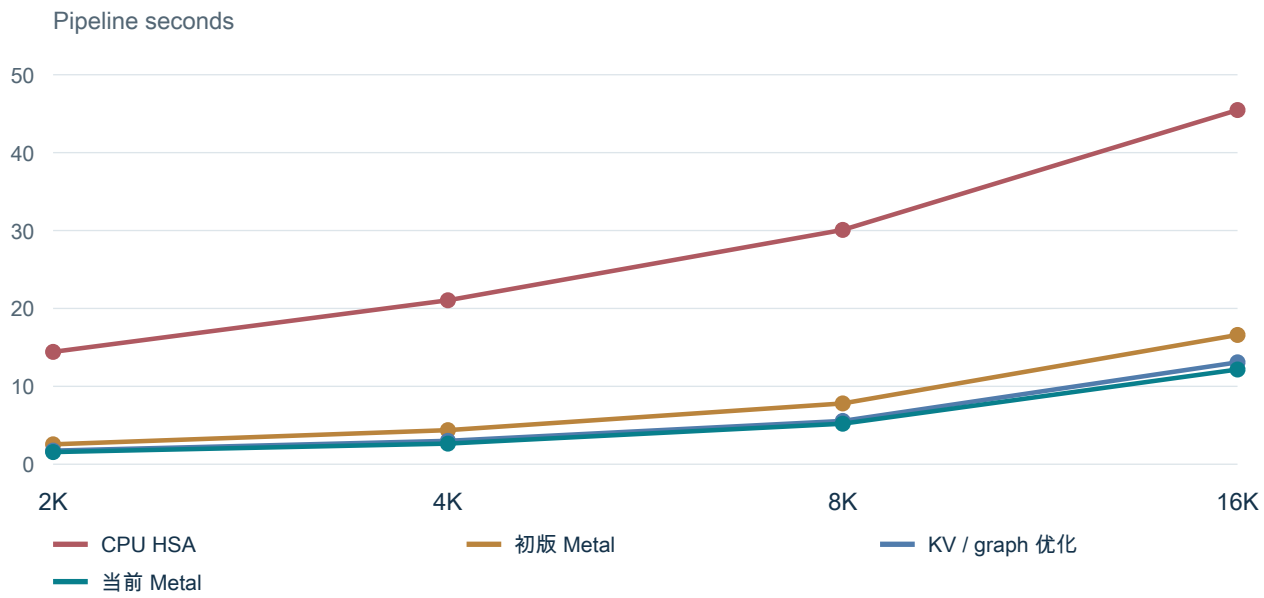
HSA 从 CPU 到 Metal

2K / 4K / 8K / 16K · pipeline 性能与算法演进 · 2026-11-08

在统一的 GPU 模型配置下，CPU HSA 到当前 Metal HSA 的整段推理加速为 3.74–9.24 倍；decode 加速为 20.67–49.65 倍。当前 HSA 仍慢于原生 Metal 全量 attention，下面同时给出该参照。

整段 pipeline：prefill + Stage 1 + HSA 初始化 + 32 步 decode

输入	CPU HSA 秒	初版 Metal 秒	KV / graph 秒	当前 Metal 秒	CPU→当前 加速
2K	14.423	2.554	1.744	1.562	9.24×
4K	21.046	4.375	3.002	2.648	7.95×
8K	30.081	7.805	5.560	5.203	5.78×
16K	45.468	16.596	13.082	12.160	3.74×



读图说明

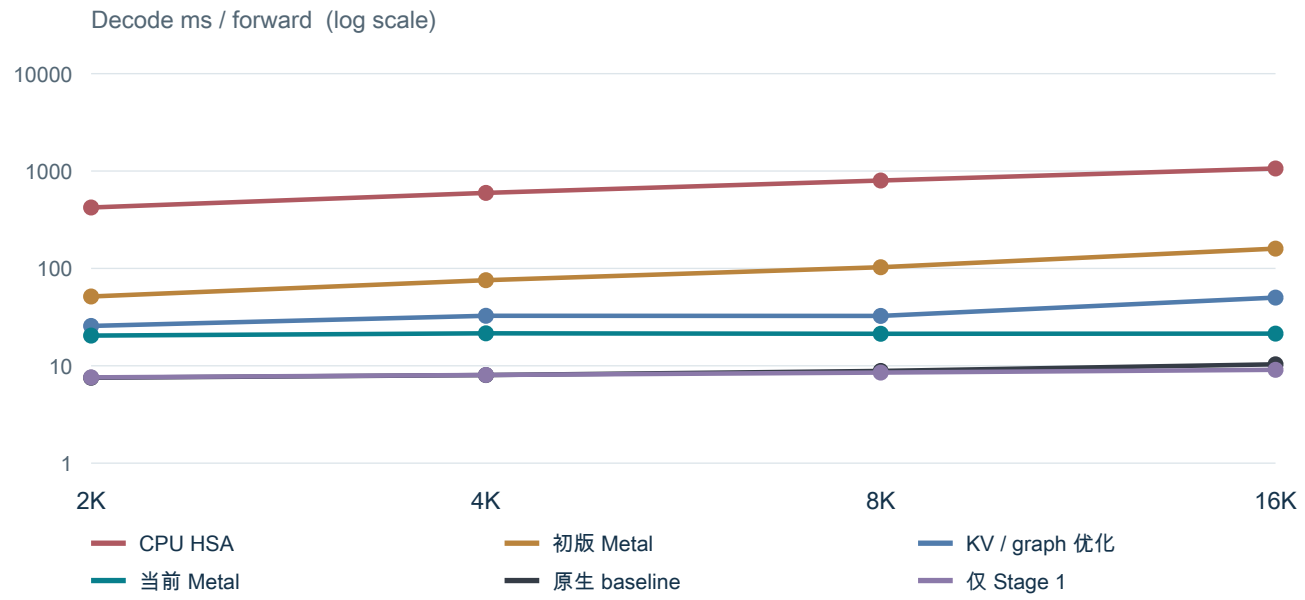
每个输入长度、每个版本只补测一次，所有版本使用同一段文本、同一组固定 token 和 32 次 decode forward。每步耗时是该单次运行内的平均值，不是多轮中位数；小幅差异不应作统计显著性结论。

pipeline 不含模型加载、进程启动、分词和末尾清理。固定 token 绕过采样，所以这是受控的请求计算耗时，不是用户实际等待的完整墙钟时间。原始数据见 [S1]。

Decode：速度提升发生在哪里

单位：ms / forward，越低越好。模型主体始终在 GPU，CPU HSA 仅指 HSA 在 CPU 执行。

输入	CPU HSA	初版 Metal	KV / graph 优化	当前 Metal	原生 baseline	仅 Stage 1
2K	422.25	51.48	25.65	20.42	7.54	7.60
4K	596.40	75.67	32.62	21.54	8.03	8.05
8K	799.08	102.82	32.52	21.32	8.85	8.52
16K	1062.37	159.68	50.16	21.40	10.35	9.09



分阶段 decode 加速倍数

输入	CPU → 初版 Metal	初版 → KV / graph	KV / graph → 当前
2K	8.20×	2.01×	1.26×
4K	7.88×	2.32×	1.51×
8K	7.77×	3.16×	1.53×
16K	6.65×	3.18×	2.34×

CPU → GPU 的大幅收益来自把近似评分、选择和精确稀疏 attention 移到 GPU；之后的收益主要来自减少数据搬运、同步和排序开销。以上计时覆盖整个模型的 decode forward，不能解释成单个 HSA kernel 的耗时。

原生 baseline 与仅 Stage 1 都使用原生 attention；HSA 还需要近似评分、Top-K 和自定义精确 attention，因此压缩 KV 或减少被选 token 并不保证 decode 更快。

整个 pipeline 的耗时组成

各组件是同一次运行的实测值，单位：秒。Decode 包含 32 次 forward 与循环管理。

输入 / 版本	Prefill	Stage 1	HSA init	Decode	合计
2K / CPU HSA	0.824	0.087	0.0000	13.512	14.423
2K / 初版 Metal	0.821	0.086	0.0000	1.647	2.554
2K / KV / graph 优化	0.823	0.100	0.0000	0.821	1.744
2K / 当前 Metal	0.821	0.087	0.0000	0.654	1.562
4K / CPU HSA	1.786	0.175	0.0000	19.085	21.046
4K / 初版 Metal	1.786	0.167	0.0000	2.421	4.375
4K / KV / graph 优化	1.784	0.174	0.0000	1.044	3.002
4K / 当前 Metal	1.785	0.174	0.0000	0.689	2.648
8K / CPU HSA	4.164	0.346	0.0000	25.571	30.081
8K / 初版 Metal	4.167	0.347	0.0000	3.290	7.805
8K / KV / graph 优化	4.169	0.350	0.0000	1.041	5.560
8K / 当前 Metal	4.174	0.347	0.0000	0.682	5.203
16K / CPU HSA	10.763	0.709	0.0000	33.996	45.468
16K / 初版 Metal	10.807	0.680	0.0000	5.110	16.596
16K / KV / graph 优化	10.791	0.685	0.0001	1.605	13.082
16K / 当前 Metal	10.787	0.688	0.0000	0.685	12.160

整段 pipeline 的分阶段耗时下降

输入	CPU → 初版 Metal	初版 → KV / graph	KV / graph → 当前
2K	82.3%	31.7%	10.5%
4K	79.2%	31.4%	11.8%
8K	74.1%	28.8%	6.4%
16K	63.5%	21.2%	7.0%

当前 16K 测试中，prefill 占整段时间约 88.7%。因此即使 decode 明显加速，短输出请求的整体收益也会被 prefill 稀释；生成更多 token 时，decode 的权重才会更大。不同版本的 prefill 小差异来自运行波动，不能归因于 HSA kernel 优化。

原生 baseline 的整段 pipeline 耗时为 2K：1.041 秒；4K：2.002 秒；8K：4.366 秒；16K：10.939 秒。它不执行 Stage 1 或 HSA，因此需要同时考虑 prefill 与额外压缩开销。

每一步的算法与实现变化

按提交演进解释：CPU reference → 初版 Metal → KV / graph 优化 → 当前 Metal。

1 将 HSA 搬到 GPU · 8b9b27a [S2]

HSA 先挑选 query 的重要维度，用 chunk 内 K 的 min/max 代表值估计注意力分数，再选 Top-K token，最后只对选中的 KV 做精确 QK、softmax 和加权求和。初版把这些步骤实现为 Metal kernels，利用 GPU 并行；但每层仍经 CPU callback 搬入数据、打包 KV，并提交和等待独立 command buffer。CPU reference 的算法代码未因此被替换。

2 减少重复工作与逐层同步 · a708c7c [S3]

直接读取原始 F16 KV，避免每步完整转换/复制为 F32 工作数组；逐层缓存 chunk 的 min/max，追加 token 时只更新尾部。可容纳的 Top-K 排序改用 threadgroup 内存。兼容的共享缓冲区路径把 HSA 编码进 GGML 现有 Metal encoder，直接读 Q/K/V、写输出，把 CPU 错误检查留到 decode 同步后，减少逐层提交与等待。该阶段是组合优化，本次未逐项拆开重测。

3 缩小排序并提高 kernel 执行效率 · dbf4743 [S4]

同一 chunk 的 token 近似分数相同，因此先排序 chunk，再展开 token ID 并按时间顺序排列，减少主排序的规模。加权求和改用 64 线程组；每次提前加载四个 V，但仍按原顺序累加。排序中 SIMD 组内交换使用 shuffle，减少共享内存访问和线程组屏障。

当前阶段的逐项收益：此前三轮交替 A/B [S5]

保留改动	8K decode 耗时下降	16K decode 耗时下降
chunk 排序并展开 token	17.77%	46.73%
加权求和：64 线程组	1.83%	2.17%
提前加载四个 V，保持累加顺序	17.93%	17.11%
SIMD shuffle 排序	1.20%	1.69%

上述百分比是各自相对上一保留版的增量，不能直接相加，也不是整段 pipeline 的下降比例。此前 8K / 16K 最终累计 decode 耗时下降分别为 34.29% / 57.44%；2K / 4K 没有逐项历史实验，本文只补测阶段组合。改变浮点归约顺序而导致模型数值漂移的方案已撤回。

实验口径、验证与复现

单轮补测用于补齐四档输入；历史三轮结果用于解释已经验证过的细分优化。

项目	设置
硬件 / 模型	Apple M2 Ultra；Llama-3.2-3B-Instruct-Q8_0
计算配置	模型全 GPU offload（99）；KV F16；16 CPU threads；budget=1024
输入	sample 201 文本重复 8 次；中间截断到 2048 / 4096 / 8192 / 16384 token；n_ctx=输入长度+1024
HSA 实际规模	四档 top-k 均为 512；chunk 大小依次为 1 / 2 / 2 / 3。输入经 Stage 1 缩减后，HSA 候选数小于原始输入长度。
输出	固定 token IDs 1000..1032；max_tokens=33；首 token 来自 prefill，后续 32 次 decode forward；trace 关闭
版本	CPU HSA 与初版 Metal：8b9b27a；KV / graph：a708c7c；当前 Metal：dbf4743
构建	历史项目源码均链接同一套当前 patched llama.cpp 库；仅临时 runner 解除 forced_tokens 对 trace 的要求
统计	每版本每长度 1 次，共 24 个有效测试；不外推为 LongBench 质量结果

验证范围

本次检查每次运行均为准确的输入长度、32 次 decode forward，HSA 执行 28 层 × 32 步并使用指定后端。当前版本此前通过完整回归、UBSan、Metal shader validation；8K / 16K / 32K 固定 4 步模型比较中，attention 和 logits 与优化前 Metal 完全一致 [S5]。这不表示 CPU 与 GPU 全模型逐位一致，也不替代生成质量评测。

数据与源码索引

[S1] docs/hsa_pipeline_benchmark.json：本次 24 次运行的完整指标、配置、输入哈希；本 PDF 所有新表格由该文件计算。

[S2] git 8b9b27a：Implement Metal HSA backend and organize project layout。

[S3] git a708c7c：Optimize Stage 2 Metal KV access and graph execution。

[S4] git dbf4743：Optimize Metal HSA selection and weighted attention。

[S5] HSA_OPTIMIZATION_LOG.md：逐项 A/B、撤回原因、正确性记录与源代码差异。

复现

在仓库根目录执行：

```
.venv/bin/python docs/hsa_pipeline_report.py bench
.venv/bin/python docs/hsa_pipeline_report.py pdf
```

PDF 生成依赖 reportlab==4.4.3 和本机 Arial Unicode 字体。bench 需要已有模型、samples.json 和编译好的 llama.cpp；已有单轮记录会复用，要全新重测可先移走 JSON。正式推理源码不变，临时构建会自动清理。